

Redis Internals & Event Loop

When you first hear about Redis, one architectural choice sounds like a terrible mistake: **Redis processes operations using a single primary thread.**

In an era where modern cloud servers have 32, 64, or 128 CPU cores, how does a single-threaded server comfortably deliver **over 100,000 operations per second** with [sub-millisecond latency](#)?

To understand Redis, you need to look inside its execution model and understand the three architectural pillars that make it work.

1. The In-Memory Architecture (RAM vs. Disk)

Traditional databases like MySQL, PostgreSQL, or MongoDB are designed around **durability first**. Their storage engines (like InnoDB) organize data inside B-Trees on persistent storage (SSD or NVMe disk).

Even with fast modern SSDs, reading a page from disk involves:

- Disk controller communication
- File system overhead
- Operating system page cache lookups

Redis inverts this priority: **memory first, persistence second.**

All keys, values, and data structures are allocated directly inside host RAM. Accessing an address in RAM requires roughly **50 to 100 nanoseconds**, compared to **50,000 to 150,000 nanoseconds** for an SSD read. By removing disk seek latency from the read path, Redis eliminates the biggest hardware bottleneck in computing before any code even runs.

2. The Illusion of Multi-Threading (Why Single-Threaded Wins)

In traditional multi-threaded servers (like standard Java applications), higher throughput comes at a steep hidden cost:

1. **Context Switching Overhead:** The operating system CPU scheduler constantly pauses one thread to run another. Saving and restoring register states, program counters, and stack pointers burns significant CPU cycles.
2. **Lock Contention & Race Conditions:** When multiple threads read and mutate shared memory, you must introduce mutexes, semaphores, or synchronized blocks. Threads end up spending significant time simply waiting for locks to release.
3. **Deadlocks & Thread Safety Bugs:** Writing error-free concurrent data access requires immense defensive programming, which introduces runtime overhead.

Redis avoids these issues entirely by running its command execution loop on a **single thread**.

Because only one thread ever reads or mutates data:

- **Zero locks:** Redis never acquires mutexes or read/write locks on keys.
- **Zero context switches:** The CPU core executing Redis spends 100% of its time executing commands rather than swapping thread state.
- **Deterministic atomicity:** Every basic command (INCR , HSET , LPUSH) is natively atomic. You never have to worry about another concurrent request corrupting an operation mid-execution.

3. Non-Blocking I/O Multiplexing

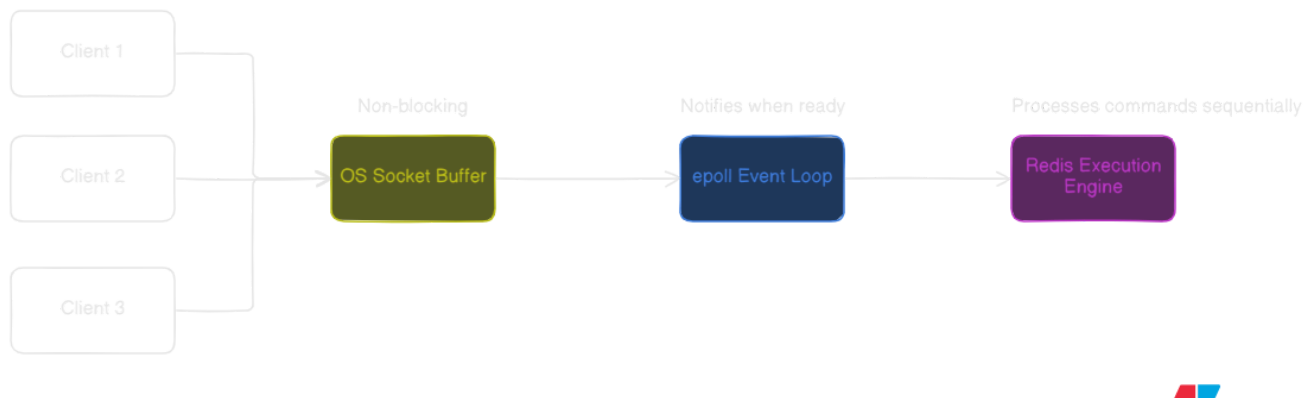
If Redis is single-threaded, what happens when 10,000 clients connect simultaneously? Wouldn't one slow network connection block every other user from getting a response?

This is where beginners get confused. **Command execution is single-threaded, but network socket handling is not blocked by slow clients.**

Redis uses **I/O Multiplexing** built on OS-level event notification primitives:

- **epoll** on Linux
- **kqueue** on macOS and BSD

How It Works



Instead of dedicating one thread per client connection, Redis registers thousands of incoming network sockets with the operating system kernel via **epoll**.

1. When a client connects or sends a payload, the kernel notifies the Redis event loop: *"Socket #42 has a complete query ready to read."*
2. Redis reads the payload from that specific ready socket, executes the command in memory, writes the result to the output buffer, and moves instantly to the next ready socket.

3. If a client is idle or network packets are delayed, Redis completely ignores that socket and continues serving other active requests.

Redis is never waiting idle on network I/O. It only processes requests that are fully staged and ready in memory.

4. Low-Level Data Structure Optimization

Redis does not store raw Java-style objects or generic representations. It uses hyper-optimized C data structures designed to minimize memory footprint and maximize CPU L1/L2 cache locality:

- **SDS (Simple Dynamic String):** Unlike standard C null-terminated strings, SDS tracks its own length. Calling `STRLEN` is $O(1)$ instead of scanning memory. It also features automatic buffer pre-allocation to eliminate frequent memory allocations on string appends.
- **ZipLists & Listpacks:** When a list, hash, or sorted set has few elements, Redis stores them as a tightly packed, contiguous block of memory instead of a linked list with pointers. This provides near-perfect CPU cache line hits.
- **SkipLists:** Used for Sorted Sets (`ZSET`), offering $O(\log N)$ search and insertion performance without the rebalancing complexity and locking overhead of traditional AVL or Red-Black trees.

5. Persistence: What Happens If the Server Restarts?

Because Redis keeps data in RAM, power loss or a crashed container would wipe everything. To prevent this, Redis offers two complementary persistence engines:

A. RDB (Redis Database Backup)

- **What it is:** Point-in-time point snapshots of your entire dataset saved to a compact binary file (`dump.rdb`).
- **How it works:** Redis calls the Unix `fork()` system call. The OS creates a child process with copy-on-write memory to dump the data to disk in the background while the main parent thread continues serving requests without interruption.
- **Trade-off:** Fast recovery, but you risk losing a few minutes of data if the server crashes between scheduled snapshots.

B. AOF (Append-Only File)

- **What it is:** A write log that logs every write command (`SET` , `DEL` , `HSET`) sequentially to an append-only log file.
- **How it works:** When Redis restarts, it replays the log from top to bottom to reconstruct state.

- **Trade-off:** Virtually zero data loss (depending on `fsync` settings: every second vs. every write), but produces larger file sizes and slower startup times.

In modern production systems, teams frequently use **hybrid persistence (RDB snapshot + AOF tail)** to balance fast startup with durability.

6. Core Mental Model

Do not ask the hardware to do complicated locking or wait for spinning disks. Keep data strictly in RAM, organize it with low-overhead C memory layouts, listen to thousands of connections simultaneously using OS event loops, and execute commands one after another at CPU memory speeds.